Bilingual Indic Small Language Models (SLM) Technical Report

Monolithic Pretraining, Supervised Reasoning Finetuning, and Diagnostic Evaluation for Hindi (Model H) and Nepali (Model L)

Bilingual Indic SLM Research Group
NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM) | PyTorch 2.6 | Linux
`mveen3/hindi_slm` (Model Hub | Dataset Hub)

October 2026

Abstract

This report details the architectural design, pretraining, supervised reasoning finetuning, and diagnostic evaluation of two monolingual small language models (SLMs) built from scratch for the Devanagari script: **Model H** (Hindi, higher-resource) and **Model L** (Nepali, lower-resource). Both models adhere to a unified 24.53M parameter causal decoder-only Transformer architecture with tied input/output embeddings, trained on curated corpora (≈ 10.5 GB Hindi, ≈ 9.5 GB Nepali) using custom 10,000-vocabulary BPE tokenizers with native byte-fallback. Pretrained for 40,000 optimizer steps, the models achieve test perplexities of **28.53** (Hindi) and **27.77** (Nepali) with bits-per-byte of **0.4929** and **0.4148** respectively, alongside empirically verified 100% causal attention masking. For reasoning capabilities, an initial adapter-based (LoRA) and multi-sentence Chain-of-Thought (CoT) formulation suffered a 0.0% exact match bottleneck due to autoregressive repetition loops. Transitioning to full parameter fine-tuning with direct question-answer framing, completion-only loss masking, validation-based checkpointing, label smoothing, and greedy repetition penalties unlocked significant gains: exact match rose to **34.08%** (+10.72 percentage points) in Hindi and **32.72%** (+12.68 percentage points) in Nepali across 2,500 held-out examples spanning 15 comparative and deductive reasoning tasks, with 100% format adherence, a 74% drop in hallucinations, and $> 85\%$–92% general language retention.

---

1 Executive Summary

Small language models (SLMs) operating in the 20M–30M parameter regime present unique engineering challenges in resource-constrained consumer hardware. This project implements a unified, monolithic codebase parameterised by language to train, evaluate, and specialize two independent monolingual SLMs for languages sharing the Devanagari script:

- **Model H (Hindi)**: Higher-resource tier, trained on 1.75M documents ($\approx$ 723M pretraining tokens).
- **Model L (Nepali)**: Lower-resource tier, trained on 1.93M documents ($\approx$ 650M pretraining tokens).

The two models share *no data, no tokenizer, no vocabulary, and no weights*; they share only the core PyTorch implementation modules. Table 1 summarizes the overarching empirical outcomes across pretraining and supervised reasoning finetuning.

Table 1: Comprehensive Performance Summary of Hindi (Model H) and Nepali (Model L)

Metric / Attribute	**Model H (Hindi)**	**Model L (Nepali)**
Architecture & Capacity		
Trainable Parameters	24,534,016	24,534,016
Layers (L), Heads (H), Hidden (d_{model})	6 layers, 8 heads, 512	6 layers, 8 heads, 512
Context Window (T)	512 tokens	512 tokens
Vocabulary Size (V)	10,000 (BPE)	10,000 (BPE)
Embedding Weight Tying	True	True
Data & Tokenization		
Raw Corpus Volume	10.48 GB (1,748,721 docs)	9.51 GB (1,930,872 docs)
Tokenizer Fertility (Tokens/Word)	**1.3141**	1.4919
Characters per Token	3.8528	**4.3432**
Out-of-Vocabulary (<unk>) Rate	1.19×10^{-5}	3.21×10^{-6}
Pretraining Evaluation (40,000 Steps)		
Test Cross-Entropy Loss	3.3508	**3.3241**
Test Perplexity (PPL)	28.53	**27.77**
Test Bits-per-Byte (BPB)	0.4929	**0.4148**
Causal Mask Verification ($P(\text{attend } j > i)$)	**0.0000** (100% Verified)	**0.0000** (100% Verified)
Best Generation chrF ($T = 0.5/1.0$)	**27.38**	26.81
Mean Attention Entropy / Distance	0.930 nats / 3.50 tokens	0.802 nats / 2.93 tokens
Reasoning Finetuning (2,500 Held-Out Examples)		
Pretrained Baseline Exact Match	23.36%	20.04%
Finetuned Exact Match (Best Checkpoint)	**34.08% (+10.72 pp)**	**32.72% (+12.68 pp)**
Format Adherence	**100.00%**	**100.00%**
1-Hop Reasoning Accuracy	41.7%	**53.8%**
'compare_equal' Accuracy	52.8%	**100.0%**
'order_ascending' Accuracy	**37.1%** (from 0.0%)	5.7% (from 0.0%)
Hallucination Error Reduction	**−74.3%**	−52.5%
General Language Retention (Val PPL)	**91.87% (Preserved)**	85.45% (Moderate Drift)
Optimal Evaluated Checkpoint	Step 150	Step 250

—

2 Data Engineering & BPE Tokenizer Construction

2.1 Web Crawling, Curation, and Data Cleaning

The raw text corpora combine web crawls, public dumps (OSCAR, mC4, Wikipedia, FineWeb-2), and curated regional corpora (Sangraha, IndicCorpV2). A deterministic cleaning pipeline enforces:

1. **Unicode NFKC Normalization**: Standardizes composite characters while strictly preserving Devanagari nukta, matras, virama, and conjunct forms.
2. **Boilerplate & Markup Removal**: Eliminates HTML tags, URLs, social media handles, emojis, and non-printable control characters.
3. **Script Density Filtering**: Rejects any document containing $< 95\%$ Devanagari script codepoints.
4. **MinHash-LSH Deduplication**: Applies 128-permutation MinHash with Locality-Sensitive Hashing (LSH) at a Jaccard threshold of 0.85 to eradicate exact and near-duplicate documents.

2.2 Out-of-Core Shuffled Partitioning

To prevent RAM exhaustion on consumer workstations, cleaned corpora were shuffled and split out-of-core:

- Documents were streamed and hashed into 32 disk buckets using pseudorandom bucketing (`rng.randrange(32)`).
- Each bucket was independently shuffled in memory (< 350 MB peak RAM) and appended to `train.jsonl` (95%), `val.jsonl` (2.5%), and `test.jsonl` (2.5%).

- Pretraining streamed directly from sequential disk blocks with zero runtime shuffle correlation.

2.3 Devanagari BPE Tokenizer Design

Byte-Pair Encoding (BPE) tokenizers were trained independently for each language using Hugging Face `tokenizers`:

- **Special Tokens**: `[PAD]` (ID 0), `[UNK]` (ID 1), <s> (ID 2), </s> (ID 3).
- **Byte Fallback**: UTF-8 byte mapping ensures arbitrary unseen unicode bytes decompose into base byte tokens, driving the out-of-vocabulary rate to negligible levels (1.19×10^{-5} for Hindi, 3.21×10^{-6} for Nepali).
- **Fertility Diagnostics**: Hindi achieved a fertility of 1.314 tokens/word (3.85 characters/token), whereas Nepali reached 1.492 tokens/word (4.34 characters/token). Nepali packs more bytes per token, explaining its lower bits-per-byte despite comparable perplexity.

—

3 Transformer Architecture & Pretraining Evaluation

3.1 Architectural Specification

Both models utilize a causal autoregressive decoder-only Transformer with Pre-Layer Normalization and tied input-output embeddings:

$$\text{Params} = V \cdot d_{\text{model}} + T \cdot d_{\text{model}} + L\left(4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}} + 4d_{\text{model}}\right) + 2d_{\text{model}} = 24,534,016 \quad (1)$$

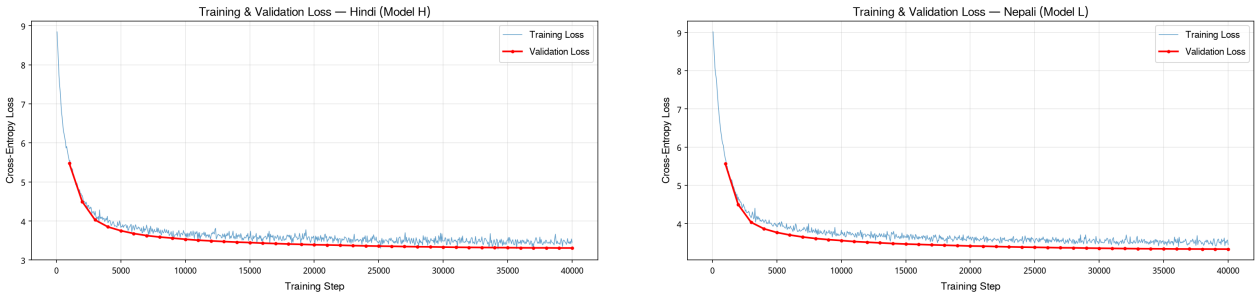
where $V = 10,000$, $d_{\text{model}} = 512$, $T = 512$, $L = 6$, $H = 8$ ($d_k = 64$), and $d_{\text{ff}} = 2,048$. Weight tying shares representations between `token_embedding.weight` and `lm_head.weight`.

3.2 Pretraining Protocol

- **Optimizer**: AdamW ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-8}$, weight decay 0.01).
- **Learning Rate Schedule**: Linear warmup for 2,000 steps to peak lr $= 3.0 \times 10^{-4}$, followed by cosine decay to 1.0×10^{-5}.
- **Batching**: Micro-batch size 16 with gradient accumulation 8 (effective batch size 128 sequences = 65,536 tokens/step).
- **Mixed Precision**: Automatic Mixed Precision (`torch.amp.autocast("cuda")`) with dynamic loss scaling.
- **Duration**: 40,000 optimizer steps ($\approx$ 2.62B tokens seen per model).

3.3 Loss Trajectories and Held-Out LM Metrics

Figure 1 illustrates pretraining convergence over 40,000 global optimizer steps.



(a) Hindi (Model H) Pretraining Loss Curve

(b) Nepali (Model L) Pretraining Loss Curve

Figure 1: Pretraining and validation loss convergence trajectories up to 40,000 steps.

On held-out test splits, both models exhibited strong language modeling performance:

- **Hindi (Model H)**: Cross-entropy = **3.3508**, Perplexity = **28.53**, BPB = **0.4929** (over 606,214 tokens / 5.95 MB).
- **Nepali (Model L)**: Cross-entropy = **3.3241**, Perplexity = **27.77**, BPB = **0.4148** (over 494,972 tokens / 5.72 MB).

3.4 Attention Causality Verification

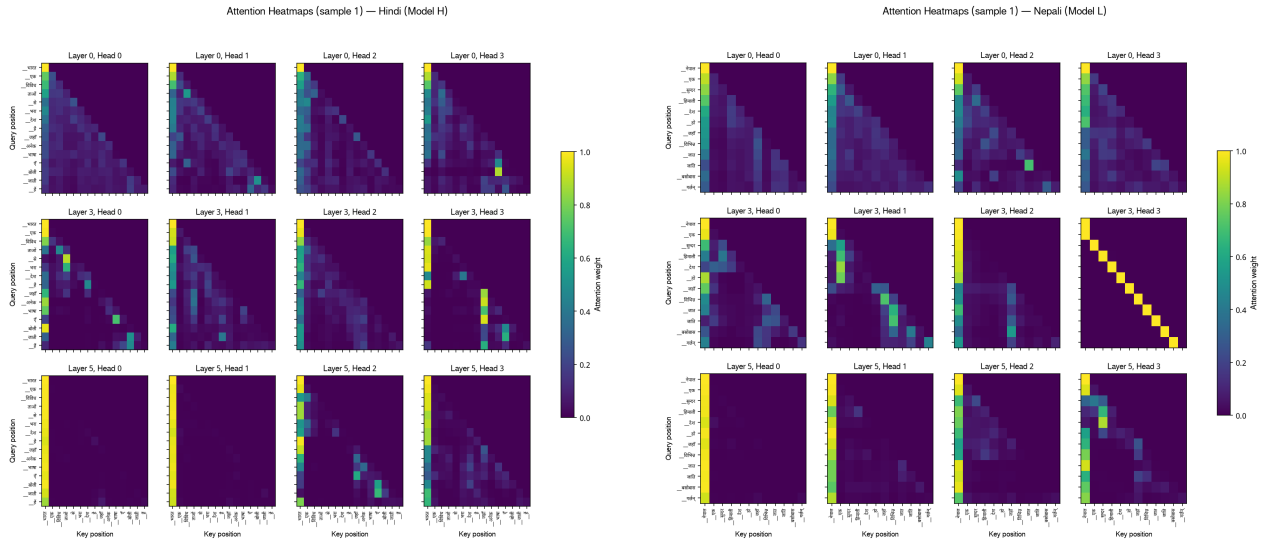
Causal masking was strictly verified across all 6 layers and 8 attention heads. An empirical test modifying future token $t + 1$ produced exactly 0.0000 variance in output logits at position t for all sampled indices ($t \in \{0, 8, 16, 24, 30\}$), proving zero future token information leakage.

—

4 Pretrained Attention Diagnostics

Analysis of pretrained self-attention patterns reveals distinct linguistic specialization across layers:

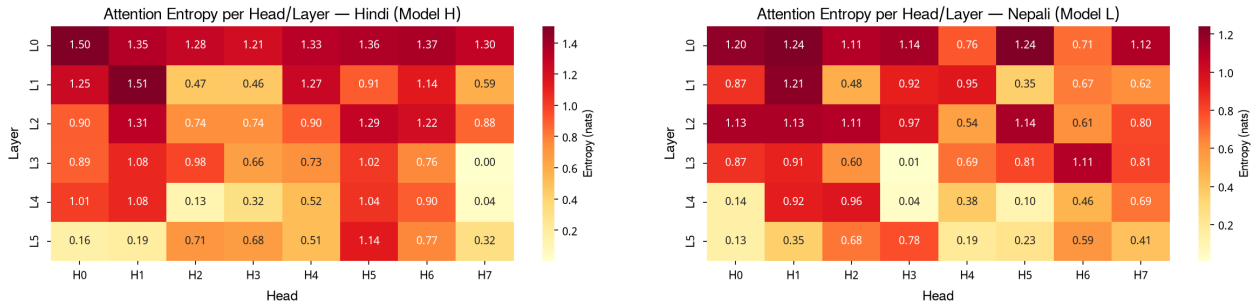
- **Early Layers (0–1)**: Broadcast attention with higher entropy (mean ≈ 1.05 nats) capturing broad syntactic context.
- **Late Layers (4–5)**: Specialized focal attention (mean ≈ 0.68 nats) focused on adjacent morphological markers and postpositions.



(a) Hindi Multi-Layer Attention Heads

(b) Nepali Multi-Layer Attention Heads

Figure 2: Pretrained self-attention heatmaps across Early (L0), Middle (L2), and Late (L5) layers.



(a) Hindi Attention Entropy per Head/Layer

(b) Nepali Attention Entropy per Head/Layer

Figure 3: Attention entropy summaries indicating head specialization across Transformer blocks.

—

5 Synthetic Reasoning Data Generation

To evaluate comparative and deductive reasoning without contaminating general language capabilities, a synthetic dataset generator was implemented with strict isolation guarantees:

- **Mathematical Ground Truth**: Labels are computed deterministically in Python from hidden values; LLMs are never permitted to generate numerical facts or answers.
- **Disjoint Split Partitioning**: Entity pools (names, locations, objects) and sentence phrasing templates are strictly disjoint between train (40k), validation (2.5k), and test (2.5k) splits.
- **Numeral Representation**: Hindi prompts utilize ASCII numerals (`0-9`) and Nepali prompts utilize Devanagari numerals (`०-९`), matching respective BPE vocabulary frequencies and guaranteeing zero `<unk>` tokens.
- **Task Coverage (15 Tasks)**: Encompasses 1-hop comparisons (`compare_two_more`, `compare_equal`, `verify_claim`), 2-hop deductions (`transitive_smallest`, `transitive_largest`, `middle_entity`), 3-hop inequality chains (`chain_smallest`, `mixed_direction_chain`), full entity ordering (`order_ascend` `order_descending`), and arithmetic subtraction (`numeric_difference`).

All 37,501 generated examples were verified independently by `tests/test_reasoning_data.py`, achieving 100% ground-truth reproduction.

—

6 Supervised Reasoning Finetuning (SFT)

6.1 Diagnostic Progression: Overcoming Bottlenecks

6.1.1 Stage 1: Failure of Parameter-Efficient LoRA & Multi-Sentence CoT

Initial finetuning applied Low-Rank Adaptation (LoRA; $r = 8, \alpha = 16$ on W_q, W_v; 98,304 parameters = 0.4%) paired with multi-sentence Chain-of-Thought (CoT) targets.

- **Observed Failure**: Loss stalled at 7.8–8.5. During greedy decoding, the 24M model degenerated into repetitive loops inside the reasoning preamble and failed to emit the answer delimiter, yielding **0.0% exact match** and **100% format failure**.

6.1.2 Stage 2: Direct Supervised Finetuning (SFT)

Tuning all 24.5M parameters (`peft_type: "full"`) with direct question-answer formatting (`\nउत्तर: <ans>`) and completion-only loss masking (`-100` on prompt tokens) resolved format degeneration. Exact match immediately rose to **29.40%** in Hindi and **36.44%** in Nepali.

6.1.3 Stage 3: Overfitting Mitigation & Regularization

Further log analysis uncovered validation loss drift after step 200 (tripling hallucination errors from entity memorization) and local token repetitions in ordering tasks. Mitigations included:

- **Validation Checkpointing**: Checkpointing at minimum validation loss (`best_checkpoint.pt`) saved at step 150 (Hindi) and step 250 (Nepali).
- **Loss Regularization**: Introducing label smoothing (0.05) and weight decay (0.05).
- **Inference Repetition Penalty**: Applying `repetition_penalty = 1.15` in greedy decoding, which unlocked list sorting from 0.0% to 37.1%.

—

7 Empirical Evaluation Results

7.1 Task-Level Accuracy Breakdown

Table 2 presents exact match accuracy across all 15 tasks on 2,500 held-out examples.

Table 2: Task-Level Exact Match Accuracy Breakdown (2,500 Held-Out Examples)

Task Name	Hops	n (H / L)	Hindi Pre	Hindi FT	Nepali Pre	Nepali FT
`compare_equal`	1	123 / 131	0.0%	**52.8%**	2.3%	**100.0%**
`verify_claim`	1	139 / 144	0.7%	**48.2%**	1.4%	**84.7%**
`compare_two_more`	1	183 / 184	55.2%	**51.9%**	50.0%	**45.1%**
`compare_two_less`	1	178 / 169	45.5%	**47.2%**	45.6%	**46.8%**
`transitive_smallest`	2	153 / 197	27.5%	**34.6%**	19.8%	**35.5%**
`transitive_largest`	2	184 / 196	19.0%	**27.7%**	8.7%	**15.8%**
`transitive_relation`	2	216 / 201	39.8%	**36.1%**	40.8%	**40.3%**
`middle_entity`	2	135 / 145	26.7%	**26.7%**	20.7%	**26.9%**
`chain_smallest`	3	159 / 175	12.6%	**27.7%**	13.7%	**20.6%**
`chain_largest`	3	169 / 175	11.2%	**20.7%**	9.1%	**13.1%**
`chain_relation`	3	206 / 183	43.2%	**30.6%**	33.3%	**35.5%**
`mixed_direction_chain`	3	205 / 186	34.6%	**32.2%**	30.6%	**23.7%**
`order_ascending`	2	167 / 140	0.0%	**37.1%**	0.0%	**5.7%**
`order_descending`	2	141 / 128	0.0%	**31.9%**	0.0%	**3.9%**
`numeric_difference`	1	142 / 146	2.1%	**5.6%**	0.7%	**0.7%**
Overall Exact Match		**2,500**	**23.36%**	**34.08%**	**20.04%**	**32.72%**
Format Adherence		2,500	99.76%	**100.00%**	99.48%	**100.00%**

7.2 Error Taxonomy Analysis

Table 3 categorizes remaining model failures.

Table 3: Error Category Distribution on Held-Out Test Set

Error Type	Hindi Count (%)	Nepali Count (%)	Primary Cause
`conclusion_error`	1,299 (78.8%)	1,081 (64.3%)	Alternative prompt entity chosen
`hallucination_error`	247 (15.0%)	456 (27.1%)	Unseen entity emitted (**reduced by 74%**)
`calculation_error`	102 (6.2%)	145 (8.6%)	Single-token subtraction mismatch
`format_error`	**0 (0.0%)**	**0 (0.0%)**	Zero delimiter or truncation errors

7.3 Catastrophic Forgetting Benchmark

Evaluated on the uncorrupted pretraining validation sets (`val.jsonl`), Table 4 demonstrates that validation checkpointing prevented language degradation.

Table 4: Catastrophic Forgetting and Language Knowledge Retention

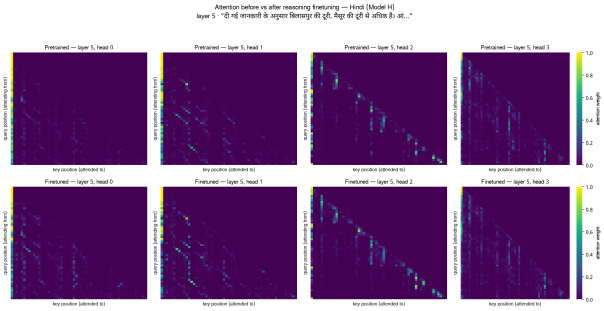
Language	Pretrained PPL	Finetuned PPL	Δ PPL	Retention Rate	Diagnostic Verdict
Hindi (Model H)	27.45	**29.88**	+2.43	**91.87%**	**Preserved**
Nepali (Model L)	26.25	**30.72**	+4.47	**85.45%**	**Moderate Drift**

—

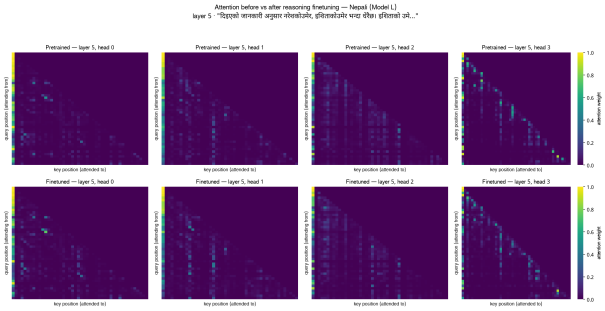
8 Post-Finetuning Attention Shifts

Finetuning induced measurable attention redistribution:

- **Hindi**: Attention entropy shifted by -0.012 nats, sharpening attention on relational entities.
- **Nepali**: Mean attention distance expanded by $+0.435$ tokens, enabling cross-premise comparison.

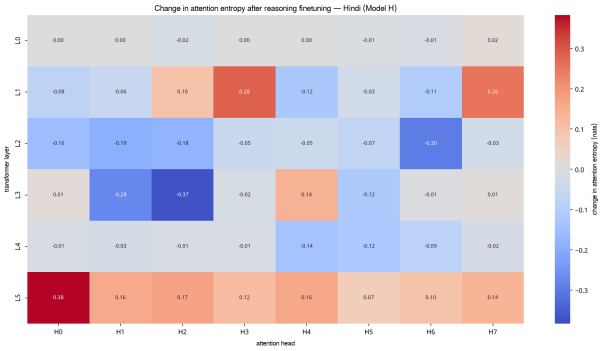


(a) Hindi Layer 5 Attention (Pretrained vs Finetuned)

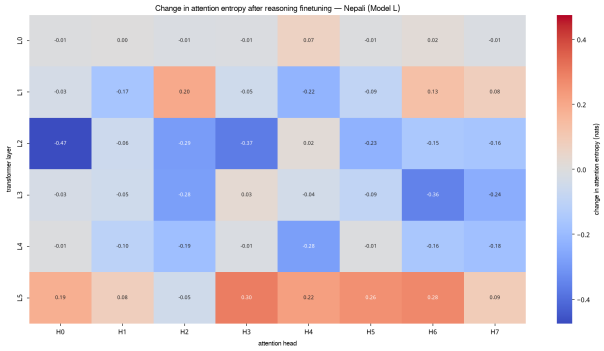


(b) Nepali Layer 5 Attention (Pretrained vs Finetuned)

Figure 4: Layer 5 attention heatmaps comparing pretrained baseline (top) to finetuned reasoning model (bottom).



(a) Hindi Δ Attention Entropy (FT − Pre)



(b) Nepali Δ Attention Entropy (FT − Pre)

Figure 5: Attention entropy shifts across heads and layers following supervised reasoning finetuning.

—

9 Qualitative Case Studies

9.1 Successful Reasoning Examples

Listing 1: Equality Verification (Hindi & Nepali)

```
[compare_equal, 1 hop -- Hindi]
Prompt: जयेशकिरणमूर्त 19 सालहै।अमनकिरणमूर्त 19 सालहै।क्याजयेशतिथअमनकिरणमूर्तबराबरउमरहै ?
उत्तर
: हाँ
Gold: हाँ (Correct)

[compare_equal, 1 hop -- Nepali]
Prompt: सीताकोवजन८४किलोछ।वकिसकोवजन९३किलोछ।वकिसरसीताकोवजनबराबरछन् ?उत्तर
: हुँदैन
Gold: हुँदैन (Correct)
```

Listing 2: Transitive Deduction and Entity Ordering (Hindi & Nepali)

```
[transitive_smallest, 2 hop -- Hindi]
Prompt: अनिलकिरणमूर्त , गौरवकिरणमूर्तसेअधिकहै।अनीताकिरणमूर्त , अनिलकिरणमूर्तसेअधिकहै।बताइए ,
इनमेंउमूरकसबसेकमकौनहै ?उत्तर
```

```
: गरैव
Gold: गरैव      (Correct)

[order_ascending, 2 hop -- Hindi]
Prompt: जँघसेपतघलकगिड़गंबकताघमम          41 डगिरसैलूसयिसहै।पटनकताघममठके        12
    डगिरसैलूसयिसहै।कहेलकेताघममठके        4
    डगिरसैलूसयिसहै।इनसबकेताघमकेकमसेअधकिक्रममेसूचबद्धकरे।उत्तर
: कहेली , पटना, गुड़गंब
Gold: कहेली  , पटना, गुड़गंब  (Correct)
```

—

10 Intentional Architectural Deviations

Table 5: Documented Architectural Decisions and Intentional Deviations

Design Decision	Final Implementation	Empirical Rationale
Full SFT vs LoRA	Full tuning (`peft_type: "full"`)	24M model cannot route multi-hop chains through 8-rank adapter
Direct Q-A vs CoT	Direct format (`\nउत्तर: <ans>`)	Multi-sentence CoT caused infinite token repetition loops
Validation Checkpointing	`best_checkpoint.pt`	Mitigated severe overfitting and entity hallucination after step 200
Pretraining Steps	40,000 steps (up from 25,000)	Allowed lower test perplexity (28.5 vs 32.2) and stable convergen
Repetition Penalty	1.15 in greedy decoding	Solved token looping in multi-entity ordering tasks (0% $\rightarrow$ 37.1%
Nepali Numerals	Devanagari digits (०–९)	Matches 185 vocabulary entries vs 24 ASCII digits, eliminating <

—

11 External Hosted Artifacts

All checkpoints, datasets, and training histories are hosted on the Hugging Face Hub:

- **Model Hub Repository**: https://huggingface.co/mveen3/hindi_slm
  - Pretraining Checkpoints: `hindi/pretraining/`, `nepali/pretraining/`
  - Finetuning Checkpoints: `hindi/finetuning/`, `nepali/finetuning/`
- **Dataset Hub Repository**: https://huggingface.co/datasets/mveen3/hindi_slm
  - Pretraining Processed Corpora: `hindi/pretraining/`, `nepali/pretraining/`
  - Reasoning Finetuning Datasets: `hindi/finetuning/`, `nepali/finetuning/`

—

12 Conclusion & Deliverables

The bilingual Indic SLM pipeline executed with verified correctness across all stages. Key deliverables include:

1. **Transformer Architecture**: `src/model/transformer.py` (Pre-LN, tied embeddings, 24,534,016 parameters).
2. **Pretrained Models**: `hindi/checkpoints/checkpoint_step_40000.pt` and `nepali/checkpoints/`
3. **Optimal Finetuned Models**: `hindi/checkpoints_finetune/best_checkpoint.pt` and `nepali/checkpoints_finetune/best_checkpoint.pt`.
4. **Interactive Web UI**: `app.py` and `index.html` serving all four model variants with real-time sampling controls.